Especificación de Requisitos de Software (ERS): TechStore

19 de octubre de 2026 · Equipo TechStore

TechStore es una tienda web de tecnología que, en su segunda entrega, pasa de HTML, CSS y JavaScript nativo a una aplicación React con Bootstrap, Atomic Design y pruebas unitarias.

1. Introducción

Este documento corresponde a la revisión 2.1 del ERS de TechStore, elaborada para la Evaluación Parcial 2 de DSY1104, Desarrollo Fullstack II. Actualiza la revisión 2.0 según el código fuente de TechStore entregado en el ZIP y mantiene la trazabilidad con la revisión 1.0 (Entrega 1).

1.1 Propósito

Definir y documentar los requisitos funcionales y no funcionales de TechStore. El documento sirve de base para el diseño, el desarrollo, las pruebas y las futuras iteraciones del proyecto, incluida la incorporación de un backend y una base de datos.

1.2 Ámbito del sistema

Nombre del sistema: TechStore.

TechStore es una tienda online de productos tecnológicos (mouse, teclados, audífonos, monitores, webcams, smartwatches, mousepads y parlantes). Se compone de dos módulos: una tienda pública orientada al cliente y un panel administrador orientado a la gestión del catálogo, los usuarios y las órdenes.

El sistema hace lo siguiente:

- Muestra un catálogo navegable con búsqueda, filtro por categoría, detalle de cada producto y una sección de ofertas.
- Permite el registro y el inicio de sesión de usuarios, con validación de RUN, correo y contraseña.
- Gestiona un carrito de compras con cantidades, total y flujo de pago con formulario de despacho.
- Registra cada compra como una orden y muestra su confirmación.
- Diferencia tres tipos de usuario (Administrador, Vendedor y Cliente) con distintos niveles de acceso.
- Permite al Administrador crear, editar y eliminar productos y usuarios, y revisar órdenes con su boleta.
- Ofrece un formulario de contacto, una página institucional y un blog informativo.

El sistema no hace lo siguiente en esta etapa:

- Procesar pagos reales mediante una pasarela de pago.
- Persistir información en una base de datos ni contar con un servidor propio; se usa `localStorage` del navegador.
- Enviar correos reales desde el formulario de contacto.

- Gestionar el stock en conexión con proveedores externos.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software
RUN	Rol Único Nacional, identificador de personas en Chile, validado con su dígito verificador en el registro
CRUD	Crear, leer, actualizar y eliminar
Stock crítico	Cantidad mínima definida para un producto; si el stock es igual o inferior, el sistema alerta al Administrador
`localStorage`	Almacenamiento del navegador que TechStore usa para sesión, carrito, productos, usuarios y órdenes
Rol	Perfil de un usuario (Administrador, Vendedor o Cliente) que determina sus permisos
React	Biblioteca de JavaScript para construir interfaces a partir de componentes
JSX	Sintaxis de React que permite escribir estructura parecida a HTML dentro de JavaScript
Componente	Pieza independiente y reutilizable de la interfaz
Props	Datos que un componente padre entrega a un componente hijo
Estado (state)	Dato que un componente recuerda y que, al cambiar, provoca que se vuelva a dibujar
Hook	Función de React, como `useState` o `useEffect`, que da acceso al estado y a otras capacidades
Vite	Herramienta que ejecuta y empaqueta el proyecto durante el desarrollo
React Router	Biblioteca que gestiona la navegación entre páginas sin recargar el sitio
Bootstrap	Biblioteca de estilos usada para la grilla responsiva y las tablas
Atomic Design	Metodología que ordena los componentes en átomos, moléculas, organismos, templates y páginas
Jasmine	Biblioteca con la que se escriben las pruebas unitarias
Karma	Herramienta que ejecuta las pruebas en un navegador
Cobertura	Porcentaje del código que se ejecuta durante las pruebas

Término	Definición
Mock	Objeto o función falsa que reemplaza a una real dentro de una prueba
Pull Request	Solicitud en GitHub para integrar los cambios de una rama a `main`

1.4 Referencias

- Repositorio del proyecto: https://github.com/ThiaraRC/TechStore
- ERS de TechStore, revisión 1.0 (Entrega 1) y Planilla de Requerimientos asociada.
- Pauta de la Evaluación Parcial 2 de DSY1104 (Duoc UC).
- Estándar IEEE 830 para la especificación de requisitos de software.
- Documentación oficial de React, React Router, Bootstrap, Vite, Jasmine y Karma.
- Documentación de `Window.localStorage`, MDN Web Docs.

1.5 Visión general del documento

La sección 2 describe el producto, sus usuarios y sus restricciones. La sección 3 detalla los requisitos funcionales y no funcionales, con códigos R.1 a R.31 para su trazabilidad. La sección 4 describe la arquitectura, las rutas, los datos y el conjunto de 20 pruebas (10 principales y 10 complementarias). La sección 5 resume la organización del equipo y el flujo de trabajo. La sección 6 registra los cambios respecto de las revisiones anteriores.

2. Descripción general

TechStore es una aplicación web de una sola página (SPA) construida con React, que se ejecuta íntegramente en el navegador y no depende de servicios externos.

2.1 Perspectiva del producto

TechStore es un producto independiente de tipo frontend. Está compuesto por dos subsistemas relacionados entre sí:

- **Tienda (público):** Inicio, Productos, Detalle de producto, Categorías, Ofertas, Nosotros, Blog, Detalle de blog, Contacto, Registro, Login, Carrito, Checkout, Compra exitosa y Pago no completado.
- **Administrador (privado):** Dashboard, mantenedor de Productos, mantenedor de Usuarios y gestión de Órdenes con su boleta, protegidos por sesión y permisos según el rol.

Cuando se incorpore un backend y una base de datos, este documento deberá actualizarse para reflejar las interfaces entre el frontend y los servicios de servidor.

2.2 Funciones del producto

- Catálogo de productos con búsqueda, filtro por categoría, detalle y ofertas.
- Carrito de compras: agregar, aumentar, disminuir, eliminar, vaciar y pagar.
- Registro de órdenes con número, fecha, estado y boleta.
- Identidad: registro, inicio y cierre de sesión.
- Roles y permisos: Administrador, Vendedor y Cliente.

- Mantenedor de productos con alerta de stock crítico.
- Mantenedor de usuarios: crear, editar y eliminar.
- Dashboard administrativo con resumen de productos, usuarios y órdenes.
- Comunicación con la empresa mediante un formulario de contacto.
- Contenido informativo: página Nosotros y blog con detalle de artículos.

2.3 Características de los usuarios

Perfil	Acceso	Conocimientos requeridos
Administrador	Acceso total al sistema: dashboard, productos, usuarios y órdenes	Manejo de PC de nivel medio; no requiere conocimientos de programación
Vendedor	Listado de productos, listado de órdenes y detalle de una orden; el resto de las opciones administrativas no se muestran ni se permiten	Manejo de PC de nivel básico
Cliente	Tienda pública: navegar, registrarse, iniciar sesión y comprar	Manejo básico de un navegador web

Usuarios de demostración incluidos en el sistema. El archivo src/data/usuarios.js define tres perfiles iniciales, y el formulario Login acepta la contraseña de prueba 1234 para cada uno.

Perfil	Correo de prueba	Contraseña
Administrador	admin@duoc.cl	1234
Vendedor	vendedor@duoc.cl	1234
Cliente	cliente@gmail.com	1234

Las cuentas de demostración se mantienen para verificar los permisos de cada rol y están protegidas contra su eliminación desde el mantenedor de usuarios.

2.4 Restricciones

- El frontend se desarrolla con React, Vite, Bootstrap y React Router, y sus componentes se organizan según Atomic Design.
- Se mantienen los colores, la hoja de estilos y la identidad visual de la primera entrega.
- Las pruebas unitarias se escriben con Jasmine y se ejecutan con Karma.
- El sistema debe funcionar en los navegadores modernos más usados (Chrome, Edge y Firefox).
- El control de versiones se realiza con Git y un repositorio público en GitHub, con una rama por integrante y `main` como rama integrada.
- No existe backend ni base de datos: la persistencia de sesión, carrito, productos, usuarios y órdenes se resuelve con `localStorage`.
- Los correos válidos se restringen a los dominios `@duoc.cl`, `@profesor.duoc.cl` y `@gmail.com`.
- Las contraseñas deben tener entre 4 y 10 caracteres.

2.5 Suposiciones y dependencias

- Se asume que el navegador tiene JavaScript y `localStorage` habilitados; de lo contrario, el carrito, la sesión y los mantenedores no funcionan.
- Se asume el uso de una versión reciente de Node.js (el proyecto se desarrolló y probó con Node.js 22) para ejecutar el proyecto y las pruebas.
- Se asume que Google Chrome está instalado, porque Karma lo usa para ejecutar las pruebas.
- Se asume que las imágenes y el video del sitio permanecen dentro del repositorio.
- Se asume que, en una etapa posterior, la persistencia pasará a un modelo cliente-servidor con base de datos relacional.

2.6 Requisitos futuros

- Integración con un backend y una base de datos relacional para reemplazar `localStorage`.
- Incorporación de una pasarela de pago real.
- Recuperación de contraseña por correo electrónico.
- Reportes y estadísticas de ventas para el Administrador.
- Gestión de stock conectada a proveedores externos.
- Cifrado de contraseñas y sesiones mediante tokens en el servidor.
- Ampliación de las pruebas unitarias a más páginas y flujos completos.

3. Requisitos específicos

Cada requisito se identifica con un código único (R.1, R.2, ...) que se mantiene respecto de la revisión 1.0 para conservar la trazabilidad. Los requisitos nuevos de esta revisión continúan la numeración a partir de R.23.

3.1 Requisitos comunes de las interfaces

Interfaces de usuario. Las interfaces son páginas web con navegación superior y un área central de contenido. Los colores y la tipografía provienen de la hoja de estilos propia de TechStore (`src/styles/estilos.css`), que conserva la identidad visual de la primera entrega. La grilla y las tablas responsivas usan clases de Bootstrap (`row`, `col-*`, `table-responsive`). La navegación entre páginas se realiza con React Router, sin recargar el sitio.

Interfaces de hardware. El sistema debe poder usarse desde un computador, una tablet o un teléfono, incluidos dispositivos con pantalla táctil. No requiere hardware adicional.

Interfaces de software. TechStore no se integra con servicios externos. Usa las siguientes herramientas y APIs:

- `Window.localStorage`, para persistir sesión, carrito, productos, usuarios y órdenes.
- React 19 y React Router, para la interfaz y la navegación.
- Bootstrap 5, para la grilla y las tablas responsivas.

Interfaces de comunicación. No aplica en esta etapa: el sistema no consume ni expone servicios web. Toda la lógica se ejecuta en el navegador.

3.2 Requisitos funcionales

Los requisitos funcionales definen las acciones que el sistema realiza al recibir información del usuario, incluida la validación de las entradas. La columna "Rev. 2.1" indica si el requisito se mantiene, se actualizó o es nuevo respecto de la revisión 1.0.

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.1	Navegación entre vistas	Cliente	El menú visible en todas las páginas permite ir a Inicio, Productos, Categorías, Ofertas, Nosotros, Blog, Contacto, Registro, Login y Carrito sin recargar el sitio	Completado	Actualizado
R.2	Registro de usuario	Cliente	Permite registrarse con nombre, apellido, RUN, fecha de nacimiento obligatoria, dirección, región, comuna, correo, contraseña y confirmación. Valida RUN sin puntos ni guion y con dígito verificador; edad mínima de 18 años; correo de dominio permitido; contraseña de 4 a 10 caracteres; coincidencia de claves y ausencia de duplicados de RUN o correo.	Completado	Actualizado
R.3	Inicio de sesión	Cliente, Vendedor, Administrador	Permite autenticarse con correo y contraseña, aplicando validaciones de formato, dominio y longitud. Incluye tres usuarios de demostración: Administrador, Vendedor y Cliente. Redirige al Administrador a /admin, al Vendedor a /admin/productos y al Cliente al inicio de la tienda.	Completado	Actualizado

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.4	Cierre de sesión	Cliente, Vendedor, Administrador	Permite cerrar la sesión activa desde el menú, eliminando los datos de sesión	Completado	Mantenido
R.5	Control de acceso por roles	Administrador, Vendedor, Cliente	ProtegerAdmin restringe las rutas /admin según la sesión y el rol. Sin sesión redirige al login; el Administrador tiene acceso total; el Vendedor solo puede consultar listados de productos y órdenes y el detalle/boleta de una orden, sin crear, editar ni eliminar; Cliente vuelve a la tienda. El menú oculta las opciones restringidas.	Completado	Actualizado
R.6	Listado de productos	Cliente	Muestra el catálogo con imagen, nombre y precio, incluyendo el precio de oferta con el original tachado cuando corresponde	Completado	Actualizado
R.7	Detalle de producto	Cliente	Muestra imagen, nombre, descripción y precio de un producto, con opción de agregarlo al carrito	Completado	Mantenido
R.8	Agregar producto al carrito	Cliente	Permite agregar productos desde el listado, el detalle, las categorías, las ofertas y los destacados del inicio, actualizando de inmediato el contador del menú. Si el producto está en oferta se usa su precio de oferta	Completado	Actualizado

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.9	Gestión de cantidades en el carrito	Cliente	Permite aumentar, disminuir (el producto se elimina al llegar a 0), eliminar un producto y vaciar el carrito, recalculando el total de forma automática	Completado	Actualizado
R.10	Finalizar compra	Cliente	Checkout solicita nombre, apellidos, correo, calle, región y comuna obligatorios; departamento e indicaciones son opcionales. Precarga datos cuando hay sesión. Valida los campos; si total es mayor que cero crea una orden local Pagada, vacía el carrito y presenta confirmación. Permite simular un pago rechazado, que conserva el carrito; si total es cero muestra pago no completado.	Completado	Actualizado
R.11	Formulario de contacto	Cliente	Permite enviar un mensaje con nombre, correo y mensaje, todos obligatorios y validados (el correo debe ser de un dominio permitido y el mensaje no puede superar 500 caracteres). Muestra un mensaje de confirmación; no se envía correo real	Completado	Actualizado
R.12	Blog informativo	Cliente	Muestra un listado de artículos con imagen, título y resumen, cada uno con acceso a su detalle. Si el artículo no	Completado	Actualizado

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
			existe, informa y permite volver		
R.13	Mantenedor de productos	Administrador	Permite al Administrador crear, editar y eliminar productos. Valida código, nombre, precio, stock, categoría y límites de texto; el stock debe ser entero no negativo y el stock crítico es opcional, pero se valida si se ingresa.	Completado	Actualizado
R.14	Alerta de stock crítico	Administrador	Advierte cuando el stock de un producto es igual o inferior a su stock crítico. La alerta se muestra en el Dashboard	Completado	Actualizado
R.15	Mantenedor de usuarios	Administrador	Permite al Administrador crear, editar y eliminar usuarios, validando RUN, datos personales, correo, fecha de nacimiento y mayoría de edad en el alta. El alta permite elegir el rol; la edición actualiza datos personales y dirección. La eliminación pide confirmación y protege las tres cuentas de demostración.	Completado	Actualizado
R.16	Selección dependiente de región y comuna	Cliente, Administrador	Actualiza las comunas disponibles según la región elegida en los formularios de registro, de usuario y de checkout	Completado	Actualizado
R.23	Gestión de órdenes	Administrador, Vendedor	Lista las órdenes registradas y permite abrir el detalle de cada una como boleta	Completado	Nuevo

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.24	Dashboard administrativo	Administrador	El Dashboard muestra resumen de productos, usuarios y órdenes, valor de ventas registradas, productos con stock crítico y las últimas cinco órdenes.	Completado	Nuevo
R.25	Categorías y ofertas	Cliente	Permite ver los productos filtrados por categoría y una página con solo los productos en oferta	Completado	Nuevo
R.26	Búsqueda y filtro del catálogo	Cliente	El catálogo permite buscar productos por texto y filtrarlos por categoría	Completado	Nuevo
R.27	Compra exitosa y pago no completado	Cliente	Después del pago exitoso se muestra la información de la orden, cliente, despacho, productos y total. Si el pago no se completa, se informa que no hubo cobro y se ofrece reintentar o volver al carrito, conservando sus productos.	Completado	Nuevo

3.3 Requisitos no funcionales

Los requisitos no funcionales describen atributos de calidad del sistema. Los códigos R.17 a R.22 se mantienen de la revisión 1.0, actualizados a la arquitectura actual, y R.28 a R.31 son nuevos.

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.17	Rendimiento en validaciones	Todos	Las validaciones de formularios y el filtrado del catálogo se ejecutan en el navegador sin recargar la página. Su tiempo de respuesta depende del dispositivo y el navegador.	Completado	Mantenido

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
R.18	Seguridad de acceso administrativo	Todos	La protección de las vistas /admin se aplica por sesión y rol: las rutas administrativas requieren inicio de sesión y el Vendedor queda restringido a consulta de productos, consulta de órdenes y detalle de orden. La autorización está implementada en el frontend.	Completado	Mantenido
R.19	Diseño responsivo	Todos	El sitio se adapta a escritorio, tablet y móvil mediante la grilla y las tablas responsivas de Bootstrap y las reglas para pantallas pequeñas de la hoja de estilos	Completado	Actualizado
R.20	Portabilidad entre navegadores	Todos	El sistema se visualiza y funciona en los navegadores modernos más usados (Chrome, Edge y Firefox); las pruebas automáticas se ejecutan en Chrome	Completado	Actualizado
R.21	Mantenibilidad del código	Equipo de desarrollo	El código se organiza en carpetas por capas de Atomic Design (`atoms`, `molecules`, `organisms`, `templates`, `pages`) y por responsabilidad (`context`, `data`, `hooks`, `utils`, `styles`, `test`)	Completado	Actualizado
R.22	Control de versiones distribuido	Equipo de desarrollo	El proyecto se mantiene en un repositorio Git público en GitHub, con una rama por integrante e integración a `main` mediante Pull Request	Completado	Actualizado
R.28	Pruebas unitarias	Equipo de desarrollo	Existen 20 casos de prueba definidos con Jasmine en siete archivos .spec.jsx, ejecutables mediante Karma: 10 casos principales que	Completado	Nuevo

N°	Requisito	Actores	Descripción	Estado	Rev. 2.1
			cubren la exigencia de la rúbrica y 10 casos complementarios para ampliar la comprobación de lógica, persistencia, validaciones y CRUD de productos.		
R.29	Informe de cobertura	Equipo de desarrollo	Karma está configurado para generar informes de cobertura de instrucciones, ramas, funciones y líneas. La cobertura de la versión actual de 20 pruebas debe tomarse de su ejecución; los porcentajes del ERS 2.0 correspondían a 11 casos anteriores.	Completado	Nuevo
R.30	Arquitectura basada en componentes	Equipo de desarrollo	La interfaz se construye con componentes React reutilizables que reciben datos por props y gestionan su estado con hooks	Completado	Nuevo
R.31	Identidad visual consistente	Todos	La aplicación conserva los colores, la hoja de estilos y el aspecto general de la primera entrega	Completado	Nuevo

Consideraciones por factor de calidad

- **Rendimiento:** al no depender de una red ni de un servidor, validaciones y filtros responden de forma inmediata (R.17).
- **Seguridad:** la protección de las rutas `/admin` se realiza en el cliente. Al no existir backend, deberá reforzarse en el servidor cuando se incorpore la base de datos (R.18).
- **Fiabilidad:** los datos de sesión, carrito, productos, usuarios y órdenes se conservan al recargar mientras el usuario no borre los datos de su navegador.
- **Disponibilidad:** como sitio sin servidor propio, depende del servicio de publicación que se utilice y de la conexión del usuario.
- **Mantenibilidad:** cada funcionalidad tiene su archivo y cada capa de componentes su carpeta, lo que permite extender el sistema sin duplicar piezas (R.21 y R.30).
- **Portabilidad:** el sistema no depende de complementos del navegador (R.20).

3.4 Otros requisitos

No se identifican otros requisitos a la fecha de esta revisión. Cualquier requisito nuevo debe agregarse a esta sección y actualizar la Planilla de Requerimientos.

4. Arquitectura, rutas, datos y pruebas

4.1 Tecnologías

Tecnología	Versión	Uso en el proyecto
React y React DOM	19	Construcción de la interfaz con componentes, props y estado
React Router	7	Navegación entre páginas y rutas dinámicas (`/productos/:id`, `/blog/:id`, `/compra-exitosa/:id`)
Bootstrap	5.3	Grilla responsiva y tablas con desplazamiento
Vite	8	Servidor de desarrollo y empaquetado (`npm run dev`, `npm run build`)
Jasmine	7	Escritura de las pruebas unitarias
Karma con webpack y Babel	6	Ejecución de las pruebas en Chrome sin ventana y medición de cobertura con Istanbul
ESLint	10	Revisión de la calidad del código (`npm run lint`)

4.2 Organización por Atomic Design

Los componentes se organizan por tamaño y responsabilidad, de modo que cada capa se construye con las anteriores y no se duplican piezas.

Capa	Componentes	Responsabilidad
Átomos	`Button`, `Input`, `Label`, `Logo`, `Price`, `ProductImage`	Elementos mínimos reutilizables
Moléculas	`ProductCard`, `CartItem`, `QuantitySelector`, `FormField`, `BlogCard`, `InfoCard`, `ResumenUsuarios`	Combinaciones de átomos con una función concreta
Organismos	`Navbar`, `Footer`, `CartList`, `CartSummary`, `ProductoForm`, `TablaUsuarios`, `HomeSection`	Bloques completos de interfaz
Templates	`MainLayout`, `AdminLayout`	Estructura general de la tienda y del panel

Capa	Componentes	Responsabilidad
Páginas	15 páginas de la tienda y 9 del panel administrador, en `pages` y `pages/admin`	Pantallas completas asociadas a una ruta

Se suman las siguientes piezas de apoyo:

Carpeta	Contenido
`context`	`CarritoContext` (carrito compartido) y `AuthContext` (sesión)
`components/guards`	`ProtegerAdmin`, que controla el acceso al panel
`hooks`	`useClaseBody`, que aplica a cada página la clase de estilo que requiere la hoja de estilos
`utils`	Funciones puras: lógica del carrito, validaciones y formato de precios
`data`	Acceso a datos en `localStorage`: productos, usuarios, órdenes, blog, regiones y comunas
`styles`	`estilos.css`, hoja de estilos de TechStore
`test`	`montar.jsx`, ayudante para dibujar componentes dentro de las pruebas

Flujo de datos. Los datos bajan de los componentes padres a los hijos mediante props y las acciones del usuario suben mediante funciones. Un ejemplo es el carrito: `Carrito` obtiene las funciones de `CarritoContext` y las entrega a `CartList`, `CartItem` y `QuantitySelector`. Al presionar "+", la función llega al contexto, cambia el estado y React vuelve a dibujar el carrito.

4.3 Rutas y control de acceso

Las rutas se definen en `App.jsx`. Las de la tienda cuelgan de `MainLayout` y son públicas; las del panel cuelgan de `AdminLayout`, envuelto por `ProtegerAdmin`.

Ruta	Página	Acceso
`/`	Inicio	Público
`/productos` y `/productos/:id`	Catálogo y detalle de producto	Público
`/categorias` y `/ofertas`	Productos por categoría y productos en oferta	Público
`/nosotros`	Nosotros	Público
`/blog` y `/blog/:id`	Blog y detalle de artículo	Público
`/contacto`	Formulario de contacto	Público

Ruta	Página	Acceso
`/login` y `/registro`	Inicio de sesión y registro	Público
`/carrito`, `/checkout`, `/compra-exitosa/:id` y `/pago-error`	Flujo de compra	Público
`/admin`	Dashboard	Administrador
`/admin/productos`	Listado de productos	Administrador y Vendedor
`/admin/productos/nuevo` y `/admin/productos/:id/editar`	Alta y edición de productos	Administrador
`/admin/ordenes` y `/admin/ordenes/:id`	Listado de órdenes y boleta	Administrador y Vendedor
`/admin/usuarios`, `/admin/usuarios/nuevo` y `/admin/usuarios/:run/editar`	Mantenedor de usuarios	Administrador

Reglas de `ProtegerAdmin`

1. Sin sesión activa, redirige a `/login`.
2. Si el usuario es Administrador, permite el acceso.
3. Si el usuario es Vendedor, permite solo el listado de productos, el listado de órdenes y el detalle de una orden; cualquier otra ruta del panel lo redirige a `/admin/productos`.
4. Si el usuario tiene otro tipo (Cliente), lo redirige a la tienda.

El menú del panel y el menú principal también ocultan las opciones que el rol no puede usar: el Vendedor no ve Dashboard ni Usuarios.

4.4 Datos y almacenamiento

Todos los datos persistentes se guardan en `localStorage` mediante dos funciones comunes, `leer` y `guardar`, definidas en `data/almacen.js`. Cada clave lleva el prefijo `tech_` para no mezclarse con otros datos del navegador. Si una lectura falla o no hay datos, el sistema usa un valor inicial en vez de detenerse.

Clave	Contenido	Datos iniciales
`tech_productos`	Catálogo: id, código, nombre, descripción, precio, stock, stock crítico, categoría, imagen, oferta y precio de oferta	8 productos de ejemplo
`tech_usuarios`	Usuarios: RUN, nombre, apellido, correo, tipo de usuario y datos de dirección	3 usuarios de demostración: Administrador, Vendedor y Cliente

Clave	Contenido	Datos iniciales
`tech_carrito`	Productos agregados al carrito con precio aplicado y cantidad	Vacío
`tech_sesion`	Usuario con sesión activa y su tipo	Sin sesión
`tech_ordenes`	Órdenes: número, fecha, estado, cliente, dirección de despacho, productos y total	Vacío

Otros datos son fijos y se incluyen en el código: los artículos del blog (`data/blogs.js`) y las regiones con sus comunas (`data/regiones.js`).

Limitaciones de esta etapa:

- Los datos existen solo en el navegador de cada usuario y no se comparten entre equipos.
- Las contraseñas de los usuarios registrados se guardan sin cifrar; esto debe corregirse al incorporar el backend.
- Si el usuario borra los datos del navegador, se pierden el carrito, las órdenes y los cambios hechos en los mantenedores.

4.5 Pruebas unitarias y cobertura

Las pruebas unitarias están escritas con Jasmine y se ejecutan con Karma mediante npm test. Cada archivo de prueba termina en .spec.jsx y se ubica junto al componente o la función que verifica. El ZIP contiene 20 casos definidos mediante it(...) en siete archivos: 10 pruebas principales que responden a lo solicitado en la rúbrica y 10 complementarias incorporadas para ampliar las verificaciones. Los casos complementarios no sustituyen los diez requeridos.

Configuración del entorno (`karma.conf.cjs`)

- Jasmine como biblioteca de pruebas y webpack para empaquetarlas, buscando todos los archivos `src/**/*.spec.jsx`.
- Babel con los presets de JavaScript moderno y de React para interpretar JSX.
- Chrome sin ventana (`ChromeHeadless`), con una sola ejecución por comando.
- Una regla de webpack (`fullySpecified: false`) que permite importar módulos sin escribir la extensión, necesaria porque `package.json` declara `"type": "module"`.
- El ayudante `src/test/montar.jsx` dibuja un componente en un contenedor de prueba dentro de un `MemoryRouter`, y ofrece `escribir()` para simular texto tecleado y `boton()` para encontrar un botón por su texto.

Uso de mocks. Se usan funciones falsas creadas con `jasmine.createSpy`, que registran si fueron llamadas, y `spyOn(Storage.prototype, ...)`, que reemplaza las lecturas y escrituras de `localStorage` por versiones controladas.

Pruebas principales (10 casos considerados para la rúbrica).

N°	Archivo	Qué verifica
1	`ProductCard.spec.jsx`	Comprueba que la tarjeta muestre nombre, precio e imagen recibidos por props.
2	`ProductCard.spec.jsx`	Comprueba que el producto en oferta muestre el precio rebajado y el original tachado.
3	`ProductCard.spec.jsx`	Comprueba que Agregar al carrito invoque onAgregar con el producto.
4	`validacionesThiara.spec.jsx`	Comprueba validaciones del registro y permite crear un cliente con datos válidos.
5	`validacionesThiara.spec.jsx`	Comprueba validaciones del Login y guarda la sesión del Administrador.
6	`validacionesThiara.spec.jsx`	Comprueba errores y corrección de datos en el formulario Contacto.
7	`Navbar.spec.jsx`	Comprueba el contador del carrito y el aviso al cerrar sesión.
8	`Home.spec.jsx`	Comprueba la portada y el enlace para ir al catálogo de productos.
9	`QuantitySelector.spec.jsx`	Comprueba que los botones + y - ejecuten sus funciones y muestren la cantidad.
10	`carrito.spec.jsx`	Comprueba agregar, aumentar, disminuir, eliminar y contar productos, incluidos productos en oferta.

Pruebas complementarias (10 casos adicionales a los exigidos).

N°	Archivo	Qué verifica
C.1	`carrito.spec.jsx`	Calcula el total desde localStorage simulado; contempla carrito ausente, errores de lectura y escritura.
C.2	`productos.spec.jsx`	Lista los ocho productos iniciales cuando no hay datos almacenados.
C.3	`productos.spec.jsx`	Busca por ID numérico o de texto y contempla productos inexistentes.
C.4	`productos.spec.jsx`	Identifica productos con stock inferior o igual al stock crítico.

N°	Archivo	Qué verifica
C.5	`productos.spec.jsx`	Crea un producto con ID nuevo, lo guarda y comprueba validaciones de sus campos.
C.6	`productos.spec.jsx`	Asigna ID 1 al crear el primer producto si la lista está vacía.
C.7	`productos.spec.jsx`	Actualiza un producto sin modificar los otros.
C.8	`productos.spec.jsx`	No modifica la lista al actualizar un ID inexistente.
C.9	`productos.spec.jsx`	Elimina el producto señalado conservando los demás.
C.10	`productos.spec.jsx`	Mantiene la lista al intentar eliminar un ID inexistente.

Resultados de cobertura. Se ejecutaron 20 pruebas unitarias mediante Jasmine y Karma, correspondientes a 10 pruebas principales establecidas para la evaluación y 10 pruebas complementarias incorporadas para ampliar la verificación de las funcionalidades del sistema.

La ejecución final obtuvo un 100 % de pruebas exitosas (20/20) y un 100 % de cobertura del código evaluado, según los resultados generados por Karma.

Medida	Porcentaje	Elementos cubiertos
instrucciones	100%	Todas la instrucciones evaluadas
ramas	100%	Todas las ramas evaluadas
funciones	100%	Todas las funciones evaluadas
Líneas	100%	Todas la líneas evaluadas

La cobertura obtenida evidencia que las pruebas unitarias ejecutan la totalidad del código incluido en el análisis de cobertura. Las pruebas complementarias permitieron ampliar los casos verificados y fortalecer la comprobación del funcionamiento de los componentes y sus validaciones.

5. Organización del equipo y flujo de trabajo

El trabajo se reparte por módulos y cada integrante desarrolla en su propia rama, que luego se integra a main. La distribución de pruebas de la tabla describe los casos presentes en los módulos: 10 principales y 10 complementarios, con un total de 20.

Integrante	Rama	Módulos principales	Pruebas
Luna Bustamante	`luna-bustamante`	Carrito, Checkout, Compra exitosa, Pago no completado, Home, órdenes y boleta del panel	3 (2 principales y 1 complementaria)

Integrante	Rama	Módulos principales	Pruebas
Rosa Medina	`rosa-medina`	Base común del proyecto, Productos, Detalle de producto y mantenedor de productos	12 (3 principales y 9 complementarias)
Thiara Rojas	`thiara-rojas`	Login, Registro, Contacto, validaciones, usuarios, mantenedor de usuarios y Dashboard	3 principales
Andrea Huenumilla	`andrea-huenumilla`	Nosotros, Blog, detalle de Blog, Categorías, Ofertas y primera versión del Dashboard	2 principales

Flujo de trabajo en GitHub

1. Cada integrante trabaja solo en su rama y no programa directamente en `main`.
2. Antes de continuar, se actualiza la rama personal con los últimos cambios de `main`.
3. Los cambios se guardan en commits pequeños con mensajes claros y se suben a la rama personal.
4. Se crea un Pull Request desde la rama personal hacia `main`.
5. Otra integrante revisa los cambios y, si funcionan y no rompen otras partes, se integra el Pull Request.
6. Después de cada integración, todas actualizan sus ramas desde `main`.

Los archivos compartidos (`App.jsx`, `main.jsx`, `Navbar.jsx`, `Footer.jsx`, `Button.jsx`, `package.json`, `karma.conf.cjs` y los estilos generales) solo se modifican tras avisar al grupo, para evitar conflictos.

6. Control de cambios respecto de la revisión 1.0

6.1 Ficha del documento

Fecha	Revisión	Autor	Modificación
06-09-2026	1.0	Equipo TechStore (Luna Bustamante, Rosa Medina, Thiara Rojas, Andrea Huenumilla)	Versión inicial del ERS para la Evaluación Parcial 1
18-10-2026	2.0	Equipo TechStore (Luna Bustamante, Rosa Medina, Thiara Rojas, Andrea Huenumilla)	Actualización a React, Bootstrap y Atomic Design; nuevos requisitos R.23 a R.31; pruebas unitarias y cobertura
19-10-2026	2.1	Equipo TechStore (Luna Bustamante, Rosa Medina, Thiara Rojas, Andrea Huenumilla)	Actualización del ERS según la versión final del proyecto TechStore.

6.2 Cambios técnicos

Aspecto	Revisión 1.0	Revisión 2.1
Tecnología del frontend	HTML5, CSS3 y JavaScript nativo, sin frameworks	React 19 con Vite, React Router y Bootstrap 5
Estructura del código	Carpetas `css`, `js`, `img`, `pages` y `admin`	Carpetas `src/components` (Atomic Design), `context`, `data`, `hooks`, `utils` y `styles`; la primera entrega se conserva en `legacy`
Navegación	Páginas HTML separadas	Aplicación de una sola página con React Router
Estilos	Hoja de estilos externa con media queries	Misma hoja de estilos, más grilla y tablas responsivas de Bootstrap
Estado compartido	Archivos JavaScript con acceso directo a `localStorage`	`CarritoContext` y `AuthContext`, con `localStorage` como respaldo
Control de acceso	Funciones de permisos en JavaScript	Componente `ProtegerAdmin` sobre las rutas `/admin`
Pruebas	No consideradas	20 pruebas en 7 archivos con Jasmine y Karma: 10 principales para la rúbrica y 10 complementarias. Reporte de cobertura configurado; las cifras de la ejecución anterior no describen este conjunto.
Control de versiones	Repositorio Git público	Ramas por integrante, Pull Request y revisión antes de integrar a `main`

6.3 Cambios en los requisitos

Requisito	Cambio
R.2 Registro de usuario	La fecha de nacimiento pasa de opcional a obligatoria y se exige mayoría de edad; se agrega la confirmación de contraseña
R.10 Finalizar compra	Se agrega un formulario de despacho validado y un estado de orden; se sustituye el folio por un número correlativo
R.11 Formulario de contacto	El correo pasa de opcional a obligatorio, con dominios permitidos; se limita el mensaje a 500 caracteres

Requisito	Cambio
R.5 y R.18 Control de acceso	El acceso del Vendedor queda limitado a productos, órdenes y detalle de orden
R.23 a R.27	Nuevos requisitos funcionales: órdenes del panel, Dashboard, categorías y ofertas, búsqueda y filtro, compra exitosa y pago no completado
R.28 a R.31	Nuevos requisitos no funcionales: pruebas unitarias, informe de cobertura, arquitectura por componentes e identidad visual consistente
Restricción de la sección 2.4	Se elimina la restricción de usar JavaScript nativo sin frameworks
R.3 y datos de prueba	Se documentan tres usuarios de demostración incluidos en el ZIP: Administrador, Vendedor y Cliente, con accesos diferenciados y contraseña demo 1234.
R.13 y R.15	Se aclaran validaciones del mantenedor de productos, alta y edición de usuarios y protección de las cuentas de prueba.
R.24 Dashboard	Se precisa el resumen de ventas y las últimas cinco órdenes, además de stock crítico.
R.28 Pruebas unitarias	Se documenta el conjunto actual de 20 casos en siete archivos: 10 principales para la evaluación y 10 complementarias.
R.29 Cobertura	Se alcanzó un 100 % de cobertura del código evaluado mediante las pruebas unitarias desarrolladas con Jasmine y Karma. Se consideran las 10 pruebas principales exigidas por la rúbrica y las 10 pruebas complementarias incorporadas posteriormente para ampliar la verificación del sistema.